

Practical Assignment 2, A&D, 2025-2026

Merry and Pippin go shopping

Merry and Pippin are two close friends who used to be crooks. They stole vegetables from farmer Maggot and got up to all kinds of other mischief. To better their lives they swore to never steal again. But unfortunately this means they have to pay for their vegetables now. They are now always looking for ways to “hack the system” and pay less.



Figure 1: Merry and Pippin stealing veggies and Merry with a broken carrot.

The two friends heard that Dutch people are a bit like them, as Dutch people always try to buy things in sales or try to haggle. A typical trick of the Dutch is the following. As you know, when you pay by cash in a Dutch supermarket, the total sum is rounded to multiples of 5 cents. (For example, when you buy a single bell pepper you have to pay 90 cents instead of the 89 cents it actually costs. On the other hand, when you buy one bag of carrots, normally costing 76 cents, you only have to pay 75 cents.) Dutch people exploit this rounding systems by reordering their products and grouping them cleverly with the checkout dividers (beurtbalkjes).

A similar rounding system is used in shops in New Zealand, where Merry and Pippin obviously live. However, they round to the nearest multiple of 10 cents instead of 5. This means that the values $0, \dots, 4$ are rounded down to 0 cents, and the values $5, \dots, 9$ are rounded up to 10 cents.

Merry and Pippin already put their vegetables on the conveyor belt at the cashier. They want to know where to put the checkout dividers in order to save some precious cents. (They are too lazy to reorder the products.) So can you solve the following problem for them? Given n products in order with costs $c_1, \dots, c_n$ and given k dividers, can you place the dividers in such a way that the total amount they have to pay is minimised?



Figure 2: One dollar

Instructions

You may work in groups of two. For this assignment, you have to hand in two things:

- Source code. We will run the code for grading.
- A report. In the report, you have to explain the algorithm and analyze it.

Source code

- You are allowed to submit a solution in either C++, Java or Python 3. If you prefer another language, please contact teaching assistant Christos Aronis asap via Discord or `christos.aronis@ru.nl` (no guarantees!).
- All code for the assignment should be in one file.
- You must write all code yourself, copying code from fellow students or the internet is considered plagiarism. You can only use libraries that come with a minimal installation of the language.
- Hand in clear and well-documented code using sensible variable and function names.

DomJudge You can test your code using DOMjudge. For each assignment, there are two problems on DOMjudge: a practice version and a grading version. The practice version shows the input and expected output so that you can debug any errors. You can submit your code to the practice version as many times as you need. If you feel confident in your solution, you can submit it to the grading version. If necessary, you can submit multiple times to the grading version as well, but only your **last** submission is saved.

Note that testing code takes quite some processing power. Hence, submitting often will cause long wait times and should be avoided. Also, the wait times will probably get longer as the deadline gets near, so be sure to start on time.

Report Besides handing in code, a report should be handed in where you explain your algorithms and analyse their correctness and runtime complexity. The report determines almost half of the final grade! We expect a clearly written high-level explanation of how your algorithm works, and a convincing analysis of correctness and asymptotic complexity. The report can have **at most 6 pages and should follow the template** provided in *template.tex* (same template as for practical 1).

Submission The deadline for sending in your solution is **December 19th**. You must submit your **report** via **Brightspace** as a group assignment for the group “Practical 2” (for which all team members need to enroll, even if you submit alone!). You must submit your **source code** via **DOMjudge**. Only one team member has to submit a solution; the names and student numbers of both team members must be mentioned in the report.

Grading Grades will be determined as follows. You may earn up to 100 points for your solution:

- 21 points for the explanation of your algorithm.
- 12 points for the correctness analysis.
- 12 points for the complexity analysis.
- 50 points for the test results.
- 5 points for the quality of the code.

If you have any questions, first contact your workgroup TA. For more difficult or technical questions, do not hesitate to contact Christos Aronis via Discord (there is a special channel practical-assignments) or via email `christos.aronis@ru.nl`. You may also contact course coordinator Jurriaan Rot via `jurriaan.rot@ru.nl`.

Input

You are given the following data (via `stdin`):

- One line with the number of products: n ($1 \leq n \leq 5000$) and the number of dividers: k ($0 \leq k \leq 50$).
- One line with all the n costs in cents: $c_1, \dots, c_n$ ($1 \leq c_i \leq 100000$). The products come in order they are put on the conveyor belt, i.e., c_1 is closest to the cashier.

Output

You should return (via `stdout`): One line with the minimal amount (in cents) Merry and Pippin have to pay to buy all the products, using at most k dividers.

Sample inputs and outputs

Example input 1:

```
8 1
1 1 1 1 1 1 1 1
```

Output:

```
0
```

Example input 2:

```
5 2
10 17 17 17 37
```

Output:

```
90
```

Example input 3:

```
7 2
25 26 11 15 16 12 4
```

Output:

```
100
```

Example input 4:

```
4 1
25 16 21 24
```

Output:

```
80
```

Example input 5:

```
6 2
2 3 3 4 4 4
```

Output:

```
10
```